Comparisons: Bar Charts, Lollipop & Cleveland Dot

Workshop 3 — Module 3

Prof.Asoc. Endri Raco

2025/26

Data Visualization for Data Scientists
UNYT Tirana

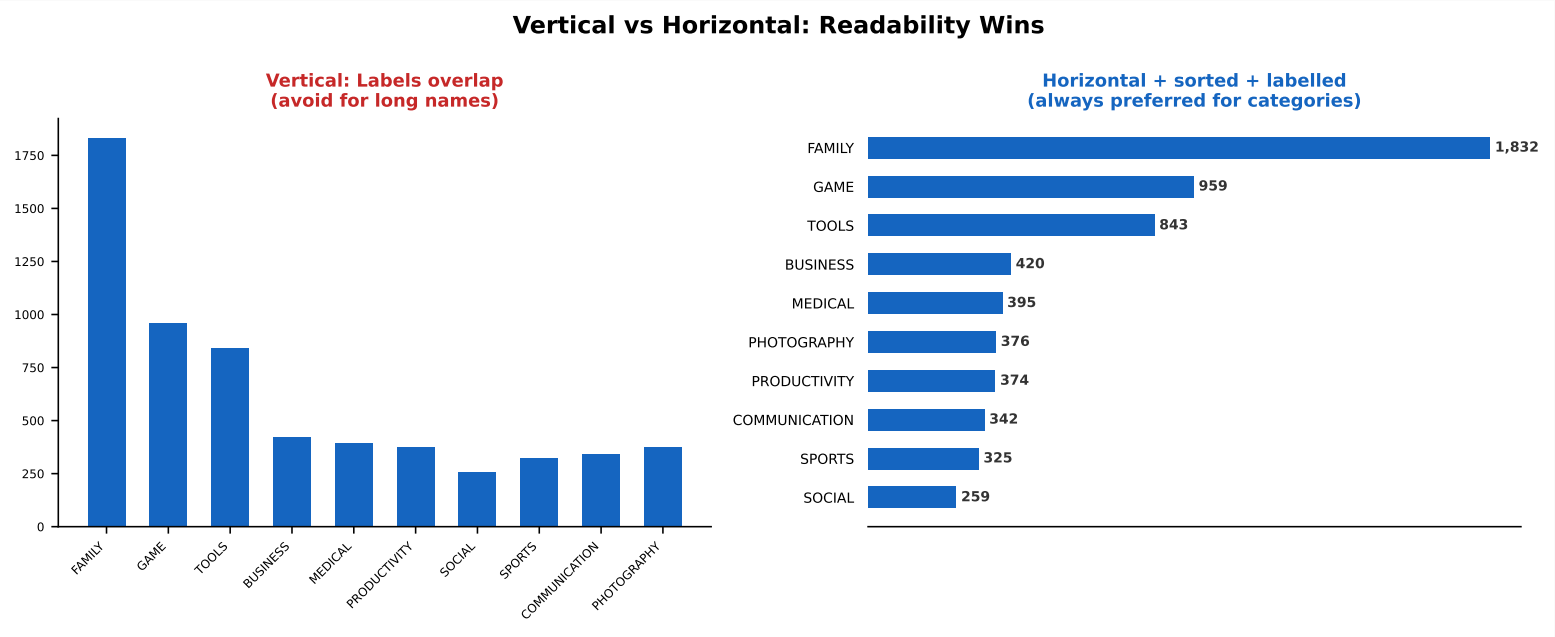
Module Roadmap

1. Bar chart best practices: horizontal, sorted, labelled
2. Three bar variants: grouped, stacked, 100% stacked
3. Lollipop charts: less ink, same precision
4. Cleveland dot plots: Tufte-minimal ranking
5. Dumbbell charts: before/after comparisons
6. Comparison chart selection guide

Learning Outcome
You will select and produce the optimal comparison chart for any categorical variable, from the standard bar chart to the Tufte-minimal Cleveland dot plot.

Bar Chart Best Practices

Vertical vs Horizontal



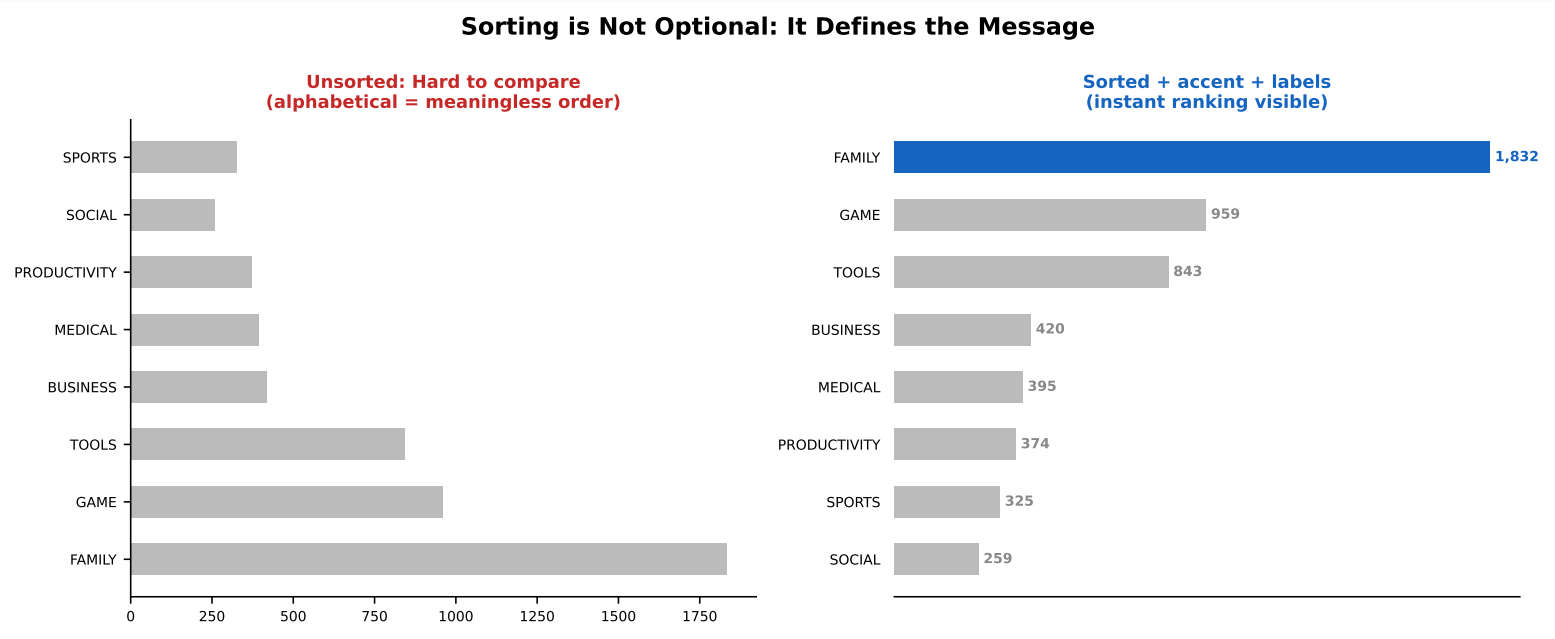
Pro-Tip

Always use horizontal bars for categorical data with text labels.

Vertical bars require rotated labels (45° or 90°), which are harder to read.

Use vertical only when x is temporal (years, months) or ordinal.

Sorted vs Unsorted



Data Insight

Alphabetical order is almost never meaningful. **Sort by value** to make ranking instantly visible. Use `fct_reorder(cat, n)` in R or `sort_values()` in Python. Add **grey-accent** colour to highlight the key finding.

Sorted Horizontal Bar in R vs Python

R (ggplot2)

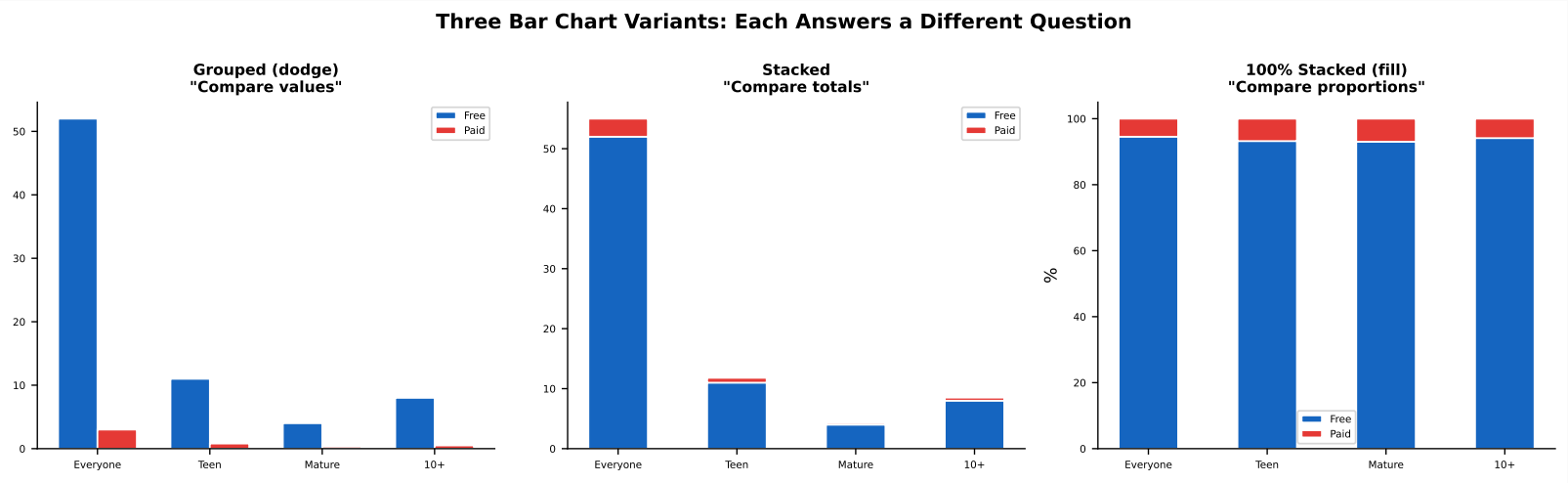
```r
apps |>
  count(Category, sort = TRUE) |>
  slice_max(n, n = 10) |>
  mutate(
    Category = fct_reorder(
      Category, n),
    hl = n == max(n)) |>
  ggplot(aes(x = Category, y = n,
             fill = hl)) +
  geom_col(width = 0.5,
           show.legend = FALSE) +
  geom_text(aes(label =
    scales::comma(n)),
    hjust = -0.15, size = 3,
    fontface = "bold") +
  scale_fill_manual(values =
    c("TRUE"="#1565C0",
      "FALSE"="#BBBBBB")) +
  coord_flip() +
  theme_minimal() +
  theme(panel.grid.major.y =
    element_blank())
```

Python

```python
top10 = (apps
    .value_counts("Category")
    .head(10).sort_values())

fig, ax = plt.subplots()
colors = ["#BBBBBB"] * 10
colors[-1] = "#1565C0"

ax.barh(top10.index, top10.values
    ,
    color=colors, height=0.55)

for i, v in enumerate(
    top10.values):
    ax.text(v + 15, i, f"{v:,}",
        va="center", fontsize=7,
        fontweight="bold",
        color="#1565C0"
          if i == 9 else "#888")

for sp in ["top","right","left"]:
    ax.spines[sp].set_visible(
         False)
ax.set_xticks([])
```

Bar Chart Variants

Grouped, Stacked, and 100% Stacked



When to Use Each Variant

Variant	Question	Limitation
Grouped (dodge)	Compare values between groups within each category	Max 3 groups; more = clutter
Stacked	Compare totals across categories + see group contribution	Hard to compare non-baseline segments
100% Stacked	Compare proportions	Loses magnitude (totals invisible)

Pro-Tip

If you have only **two groups**, use a grouped bar — proportions are visible without 100% stacking. If proportions are the message, prefer 100% stacked. If totals matter, use regular stacked.

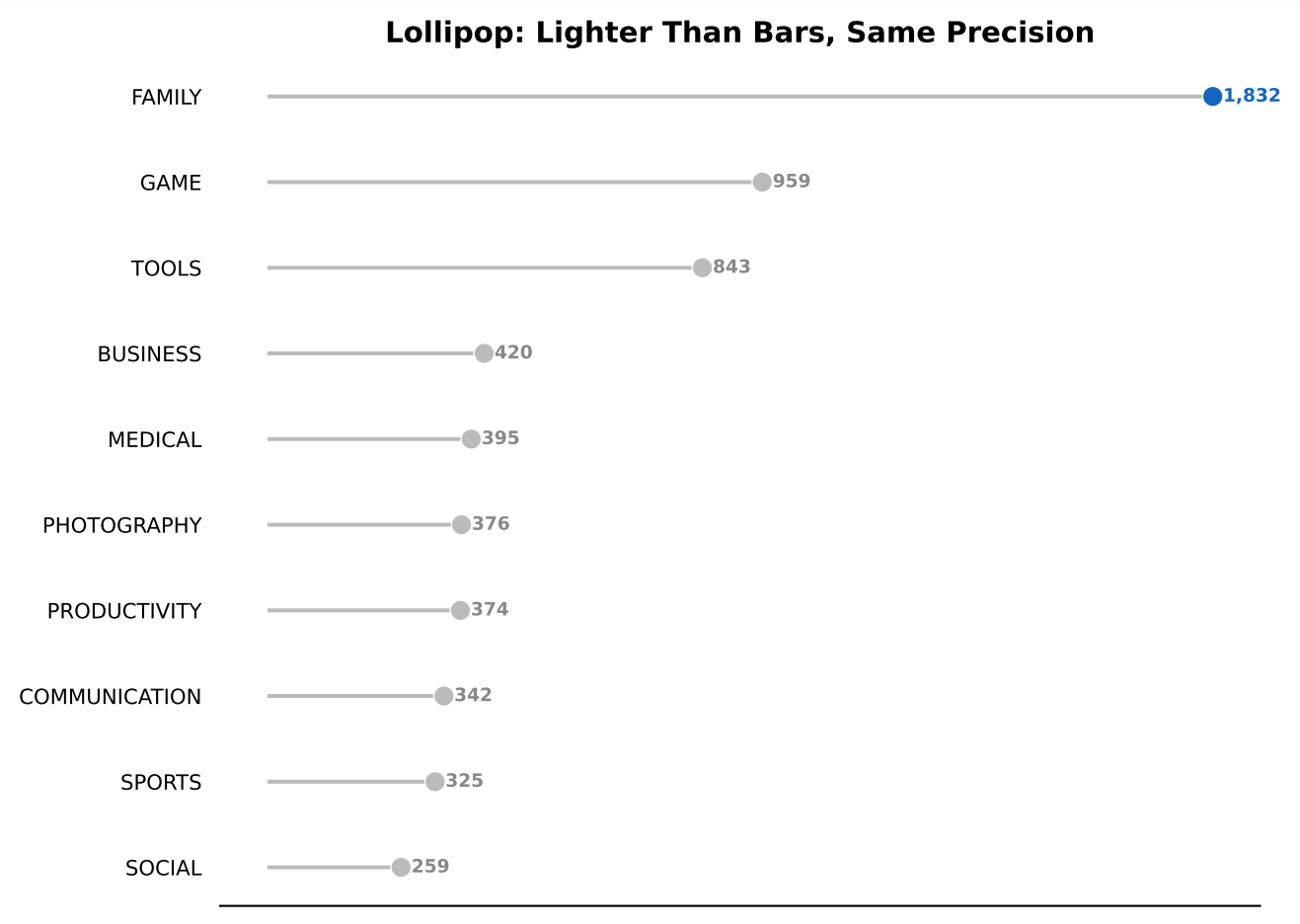
Bar Variants in R vs Python

```r
# Grouped
ggplot(df, aes(x = cat,
    fill = group)) +
  geom_bar(position = "dodge")

# Stacked
ggplot(df, aes(x = cat,
    fill = group)) +
  geom_bar(position = "stack")

# 100% Stacked
ggplot(df, aes(x = cat,
    fill = group)) +
  geom_bar(position = "fill") +
  scale_y_continuous(
    labels = scales::percent)
```

```python
# Grouped (manual offset)
x = np.arange(len(cats))
ax.bar(x - w/2, free, w,
    label="Free")
ax.bar(x + w/2, paid, w,
    label="Paid")

# Stacked
ax.bar(cats, free, label="Free")
ax.bar(cats, paid, bottom=free,
    label="Paid")

# 100% Stacked (compute %)
ct = pd.crosstab(
    df["cat"], df["group"],
    normalize="index") * 100
ct.plot.bar(stacked=True, ax=ax)
```

Lollipop Charts

Lollipop: Less Ink, Same Precision



Data Insight

The lollipop chart replaces the bar's filled rectangle with a thin line (segment) and a point. This reduces data ink while preserving the position encoding. It is especially effective for long category lists (10–20

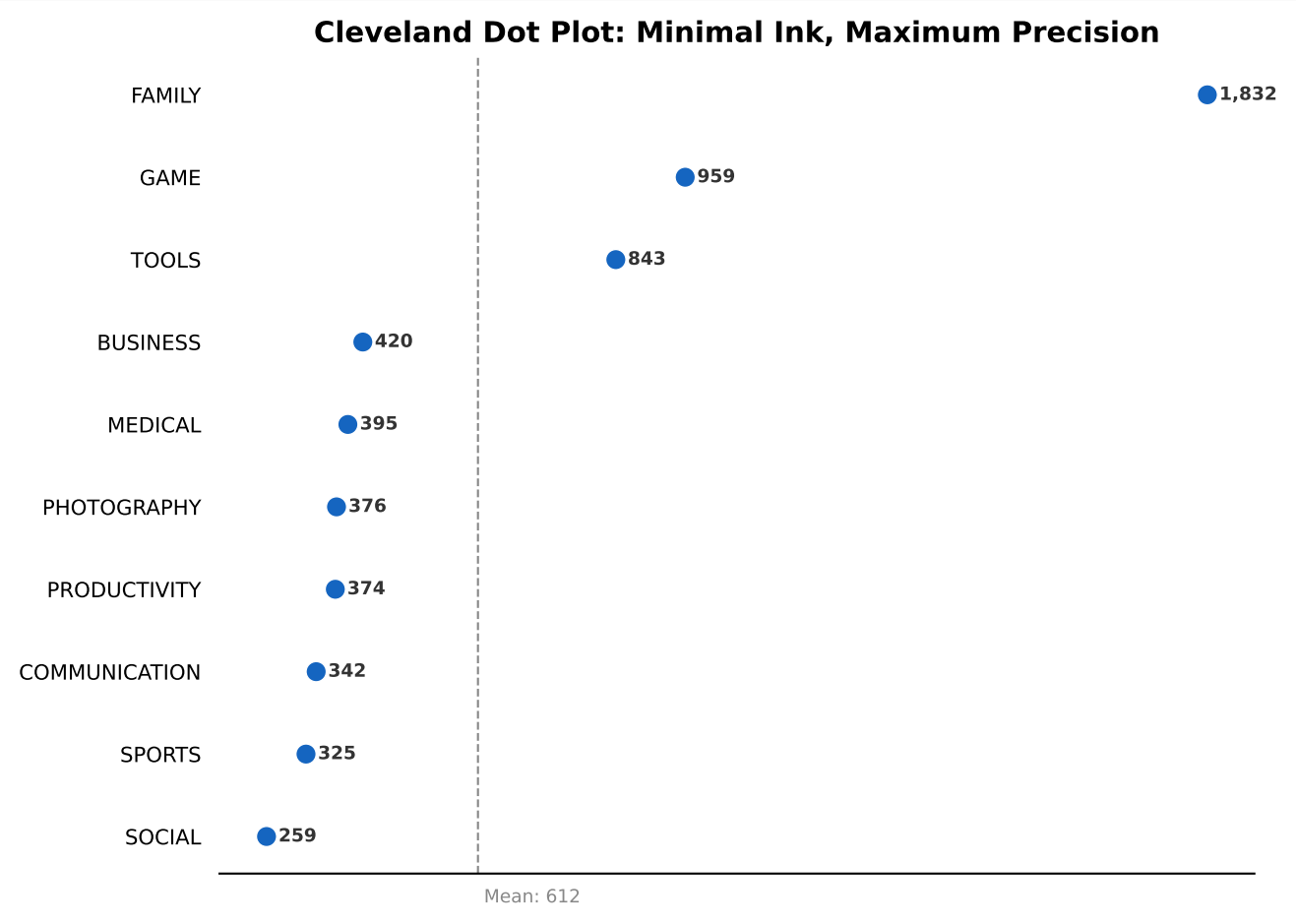
Lollipop in R vs Python

```r
ggplot(df, aes(
    x = fct_reorder(cat, val),
    y = val)) +
  geom_segment(aes(
    xend = cat, y = 0,
    yend = val),
    color = "grey70",
    linewidth = 0.8) +
  geom_point(size = 3,
    color = "#1565C0") +
  coord_flip() +
  theme_minimal() +
  theme(
    panel.grid.major.y =
      element_blank()) +
  labs(x = NULL, y = "Count")
```

```python
fig, ax = plt.subplots()

# Segments (stems)
ax.hlines(y=range(len(cats)),
    xmin=0, xmax=vals,
    color="#BBBBBB", lw=1.5)

# Points (dots)
ax.scatter(vals,
    range(len(cats)),
    s=60, c="#1565C0",
    zorder=5,
    edgecolors="white", lw=0.5)

# Direct labels
for i, v in enumerate(vals):
    ax.text(v + 10, i, f"{v:,}",
        va="center", fontsize=7,
        fontweight="bold")

ax.set_yticks(range(len(cats)))
ax.set_yticklabels(cats)
```

Cleveland Dot Plot

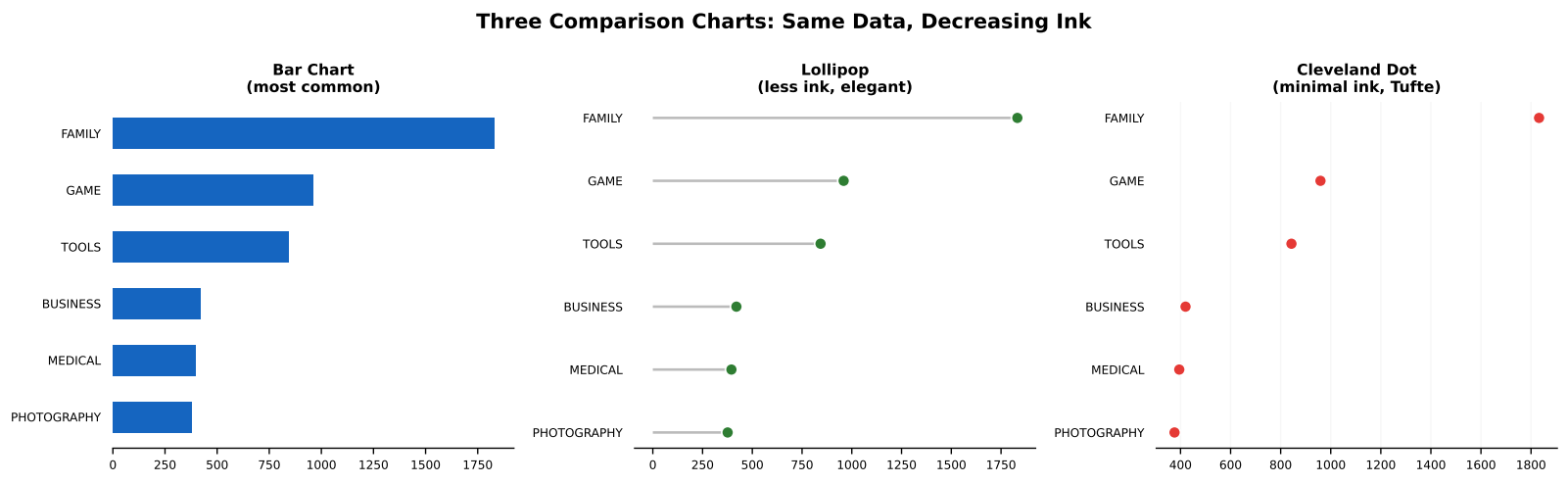
Cleveland Dot: Minimal Ink, Maximum Precision



Pro-Tip

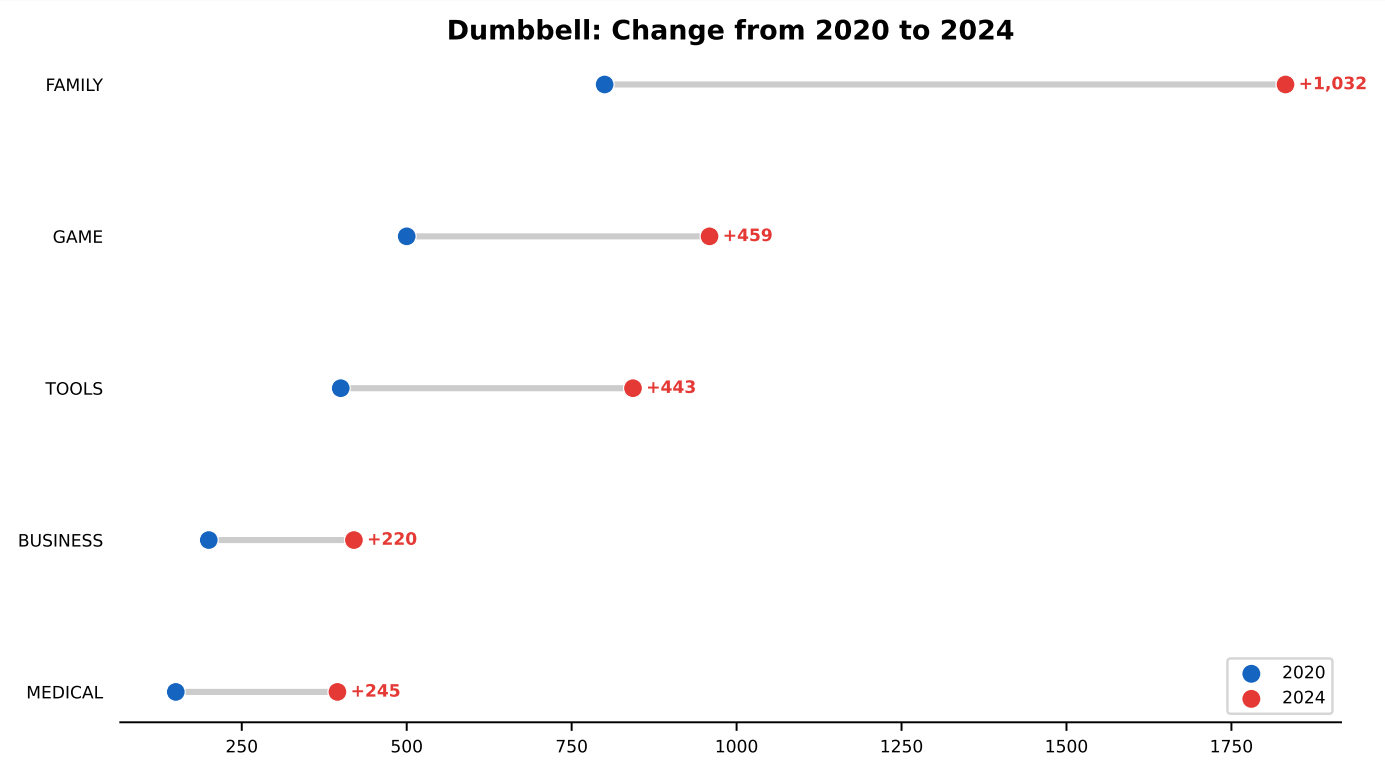
The Cleveland dot plot uses only a point per category — the absolute minimum of data ink (Tufte, 1983). Add subtle gridlines for alignment and a mean reference line for context. Best for audiences who value

Bar vs Lollipop vs Cleveland: Ink Reduction



Dumbbell Charts

Dumbbell: Before/After Comparison



Data Insight

Dumbbell charts show change between two time points per category. The connector length encodes the magnitude of change; colour distinguishes the two periods. Direct-label the change value (+459) to avoid requiring mental arithmetic.

Dumbbell in R vs Python

```r
ggplot(df, aes(
    y = fct_reorder(cat,
      val_2024))) +
  geom_segment(aes(
    x = val_2020, xend = val_
        2024,
    yend = cat),
    color = "grey80",
    linewidth = 1.5) +
  geom_point(aes(x = val_2020),
    size = 3, color = "#1565C0")
        +
  geom_point(aes(x = val_2024),
    size = 3, color = "#E53935")
        +
  geom_text(aes(x = val_2024,
    label = paste0("+",
      val_2024 - val_2020)),
    hjust = -0.3, size = 3,
    color = "#E53935") +
  theme_minimal() +
  labs(x = "Count", y = NULL,
    title = "Change: 2020 vs 2024
        ")
```

```python
for i in range(len(cats)):
    # Connector
    ax.plot([v2020[i], v2024[i]],
        [i, i], color="#CCCCCC",
        lw=2.5,
        solid_capstyle="round")
    # Start point (2020)
    ax.scatter([v2020[i]], [i],
        s=60, c="#1565C0", zorder
            =5,
        edgecolors="white", lw
            =0.5)
    # End point (2024)
    ax.scatter([v2024[i]], [i],
        s=60, c="#E53935", zorder
            =5,
        edgecolors="white", lw
            =0.5)
    # Change label
    ax.text(v2024[i] + 20, i,
        f"+{v2024[i]-v2020[i]:,}"
            ,
        va="center", fontsize=7,
        fontweight="bold",
        color="#E53935")
```

Chart Selection

Comparison Chart Selection Guide

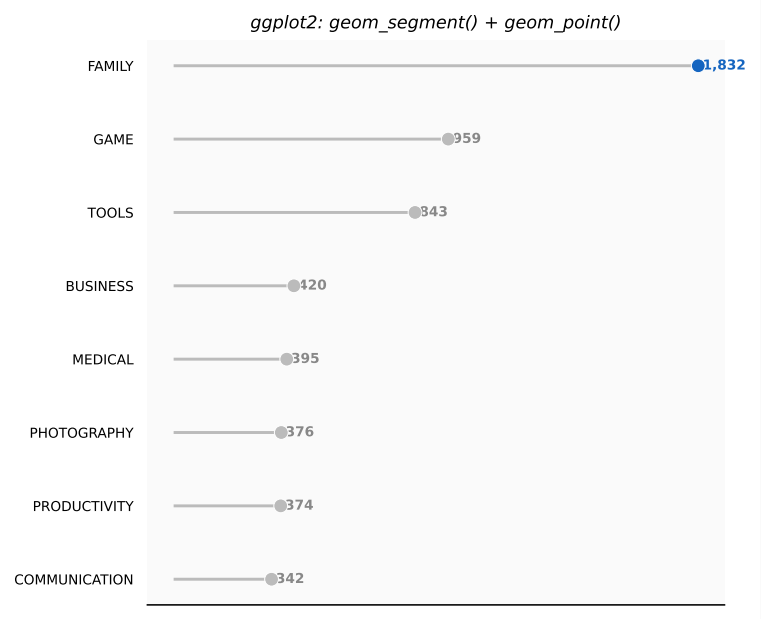
Comparison Chart Selection Guide

Need	Chart	Guidelines
Compare categories	**Horizontal bar**	Sorted, direct labels, accent on key
Compare 2 groups per category	**Grouped bar (dodge)**	Max 3 groups to avoid clutter
Part-to-whole	**Stacked bar / 100%**	Use 100% for proportions, stacked for totals
Rank with less ink	**Lollipop**	segment + point, sorted
Minimal-ink rank	**Cleveland dot**	Point only + gridlines, Tufte-approved
Before/after change	**Dumbbell**	Two points + connector per category

Complete Examples

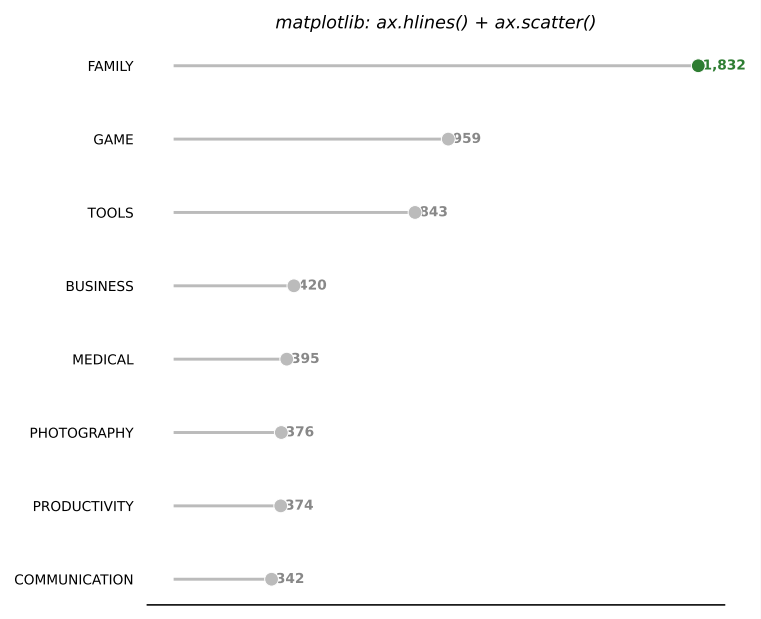
R: Publication-Quality Lollipop

```r
apps |>
  count(Category, sort = TRUE) |>
  slice_max(n, n = 8) |>
  mutate(Category =
    fct_reorder(Category, n),
    hl = n == max(n)) |>
  ggplot(aes(x = Category,
             y = n)) +
  geom_segment(aes(xend = Category,
    y = 0, yend = n),
    color = "grey75") +
  geom_point(aes(color = hl),
    size = 3, show.legend = FALSE)
       +
  scale_color_manual(values =
    c("TRUE"="#1565C0",
      "FALSE"="#BBBBBB")) +
  geom_text(aes(
    label = scales::comma(n)),
    hjust = -0.3, size = 3,
    fontface = "bold") +
  coord_flip() +
  theme_minimal() +
  labs(title = "FAMILY Leads",
       x = NULL, y = NULL)
```



Python: Same Lollipop

```python
top8 = (apps
    .value_counts("Category")
    .head(8).sort_values())

fig, ax = plt.subplots(
    figsize=(6, 5))
colors = ["#BBBBBB"] * 8
colors[-1] = "#2E7D32"

ax.hlines(y=range(8), xmin=0,
    xmax=top8.values,
    color="#BBBBBB", lw=1.5)
ax.scatter(top8.values,
    range(8), s=50, c=colors,
    zorder=5, edgecolors="white")

for i, v in enumerate(
    top8.values):
    ax.text(v + 15, i, f"{v:,}",
        va="center", fontsize=7,
        fontweight="bold",
        color="#2E7D32"
          if i == 7 else "#888")

ax.set_yticks(range(8))
ax.set_yticklabels(top8.index)
```



Synthesis

Key Takeaways

1. **Always horizontal + sorted + labelled** for categorical bar charts. Vertical only for temporal x-axes.
2. Three bar variants serve different questions: **grouped** (compare values), **stacked** (compare totals), **100% stacked** (compare proportions).
3. **Lollipop** = segment + point: less ink than bars, same precision. Best for 10–20 categories.
4. **Cleveland dot** = point only + gridlines: minimum ink, Tufte-approved. Best for precision-oriented audiences.
5. **Dumbbell** = two points + connector: shows change between two time periods per category.
6. The progression bar $\rightarrow$ lollipop $\rightarrow$ dot is a data-ink reduction cascade.

Essential Readings

- Cleveland, W. S. (1985). *The Elements of Graphing Data*. Wadsworth. (Origin of the dot plot.)
- Schwabish, J. (2021). *Better Data Visualizations*, Chapter 4 (Bar Charts and Alternatives).
- Wickham, H. (2016). *ggplot2*, Chapter 3 (Bar Charts).
- Wilke, C. O. (2019). *Fundamentals of Data Visualization*, Chapter 6 (Amounts).

Next Module: Relationships — Scatter, Bubble & Hexbin (W03-M04)